

# FPGA Accelerator for Directed Percolation

## Monte Carlo Simulation on Artix-7 at 100 MHz

Leonardo Pieripolli, Alessio Tuscano

Programmable Hardware Devices

August 15, 2026

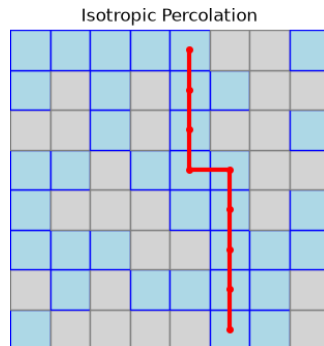
# Outline

- 1 Percolation Theory
- 2 System Architecture
- 3 Validation
- 4 FPGA Engineering Results
- 5 Physics Results
- 6 Conclusions
- 7 Backup

# What is Percolation?

**Physical picture:** a fluid filters through a porous medium.

- 2D grid: each site is **open** (blue, prob.  $p$ ) or **closed** (gray,  $1 - p$ )
- A **spanning cluster** connects top  $\rightarrow$  bottom
- **Phase transition** at critical probability  $p_c$
- Below  $p_c$ : only finite clusters  
Above  $p_c$ : spanning cluster emerges
- **Isotropic percolation:** fluid can move in all 4 directions



# Directed Percolation (DP)

**Key difference:** flow is **directional** — only downward propagation.

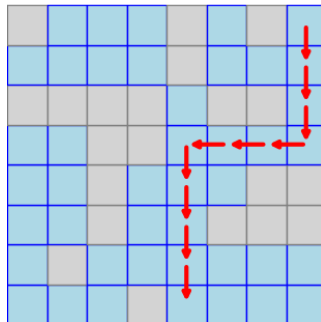
- Fluid moves only  $\downarrow$  (not  $\uparrow$ )
- Horizontal spread still allowed ( $\leftrightarrow$ )
- Different universality class from isotropic percolation
- DP critical exponents:

$$\beta = 0.2765 \quad (\text{order parameter})$$

$$\nu_{\parallel} = 1.096 \quad (\text{corr. length, time})$$

$$\nu_{\perp} = 0.733 \quad (\text{corr. length, space})$$

Directed Percolation



# Finite-Size Scaling

For a finite grid of size  $N \times N$ , the spanning probability obeys:

$$P_{\text{span}}(p, N) = F\left((p - p_c) N^{1/\nu}\right)$$

where  $F$  is a **universal scaling function**.

## Consequences:

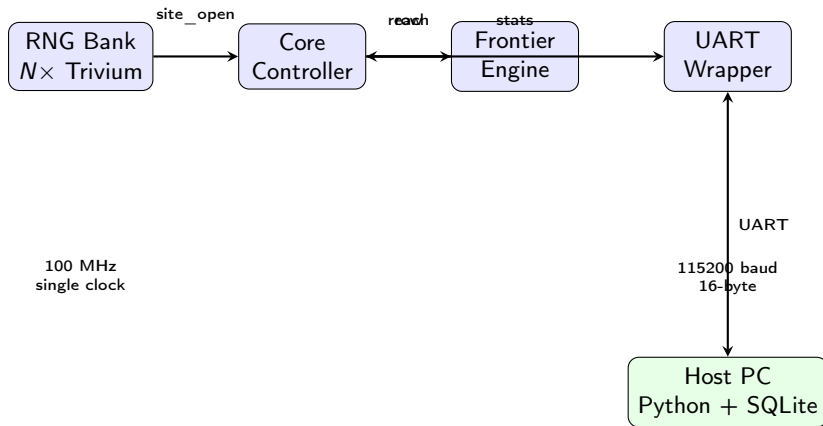
- Curves for different  $N$  **collapse** onto a single master curve
- The collapse validates the universality class
- The crossing point of  $P_{\text{span}} = 0.5$  gives an estimate of  $p_c$

**Threshold estimation** via logistic regression:

$$P_{\text{span}}(p) = \frac{1}{1 + e^{-a(p-p_c)}}$$

Fitted via maximum likelihood (binomial bootstrap for CIs).

# Top-Level Block Diagram



**Key principle:** each block is independent and pipelined.

## RNG Bank — $N_{\text{rows}} \times$ Trivium Stream Cipher

- $N_{\text{rows}}$  **independent** Trivium stream ciphers, one per grid column
- $N_{\text{rows}}$  is a **compile-time generic** ( $N_{\text{ROWS\_G}}$ ), default 64
- Synthesized for 64, 128, or 180 — same throughput, different resource usage
- **AES-CTR seeding** (one-time,  $2 \times N_{\text{rows}} \times 10$  cycles):
  - $2 \times N_{\text{rows}}$  AES blocks  $\times$  10 cycles/block
  - For  $N_{\text{rows}} = 64$ :  $128 \times 10 = 1280$  cycles
- **Trivium warmup**: 1152 bit-cycles / 32 bit/cycle = 36 cycles
- **Total warmup** ( $N_{\text{rows}} = 64$ ):  $\approx 1316$  cycles  $\approx 13.2 \mu\text{s}$  at 100 MHz

### Per cycle:

- $N_{\text{rows}}$  random 32-bit words
- Compared to threshold  $p$  (UQ32 fixed-point)
- Output:  $N_{\text{rows}}$ -bit `site_open` vector

### Validation:

- Occupancy bias  $< 0.001$
- Uniform distribution verified
- No correlation between columns

## Frontier Engine — Row-Wise Reachability

**Problem:** determine if a top-to-bottom path exists without storing the full grid.

**Algorithm (per row):**

1 **Vertical seed:**

$$\text{seed}[i] = \text{open}[i] \wedge \text{prev\_reach}[i]$$

(Row 0: seed = open)

2 **Horizontal closure** (iterative  $\pm 1$  expansion):

$$\text{reach}[i] = \text{seed}[i] \vee (\text{open}[i] \wedge (\text{reach}[i-1] \vee \text{reach}[i+1]))$$

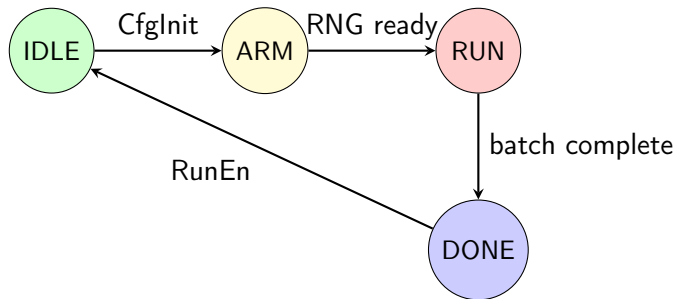
Converges in  $\leq N_{\text{rows}}$  iterations (typically 2–3 for  $N = 64$ )

**VHDL implementation:** bidirectional associative prefix scan

- **3-stage pipeline:** 3 cycles/row (READY  $\rightarrow$  COMPUTE  $\rightarrow$  SAVE)
- **4 cycles/row end-to-end:** the core's registered row-send handshake adds 1 cycle/row
- Only 2 rows of state (current + previous reachability)
- Validated against BES: 1000 random tests, exact match



## Core Controller — State Machine



### Batch processing:

- CfgRuns runs executed autonomously — no host intervention
- Accumulators (32-bit): StepCount, SpanningCount, TotalOccupied, SpanningOccupied
- RNG runs continuously — no cycle wasted waiting for random numbers

# UART Protocol — 16-Byte Frames

## Request (host → FPGA):

| Word | Field          | Purpose         |
|------|----------------|-----------------|
| 0    | CfgP (UQ32)    | Probability $p$ |
| 1    | CfgSeed        | RNG seed        |
| 2    | CfgStepsPerRun | Grid height     |
| 3    | CfgRuns        | Batch size      |

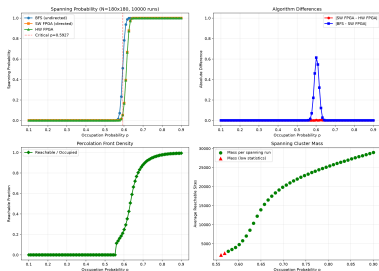
## Response (FPGA → host):

| Word | Field            | Purpose              |
|------|------------------|----------------------|
| 0    | StepCount        | Runs completed       |
| 1    | SpanningCount    | Spanning runs        |
| 2    | TotalOccupied    | Total occupied sites |
| 3    | SpanningOccupied | Reachable sites      |

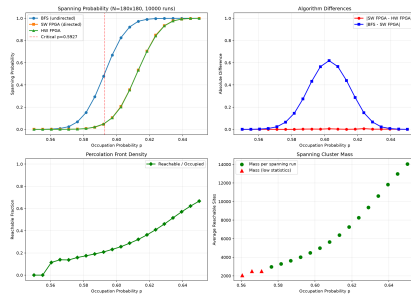
## Details:

- 115200 baud, 8N1
- Wire time:  $\approx 2.78$  ms
- Linux termios raw mode
- Fixed-point:  $p = \text{CfgP} / 2^{32}$

# Three-Way Comparison



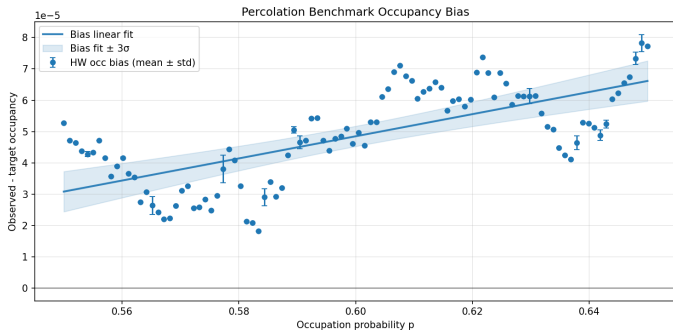
20 points, 1000 runs each



100 points, 1000 runs each

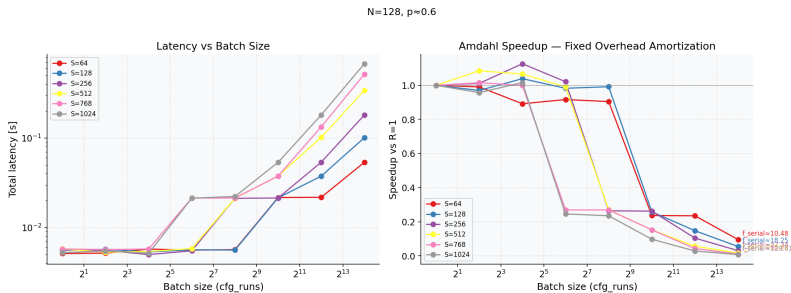
- BFS (undirected) vs SW FPGA (directed) vs HW FPGA
- Directed percolation threshold is **higher** ( $p_c \approx 0.605$ ) than isotropic ( $p_c \approx 0.593$ )

# Occupancy Bias — RNG Accuracy



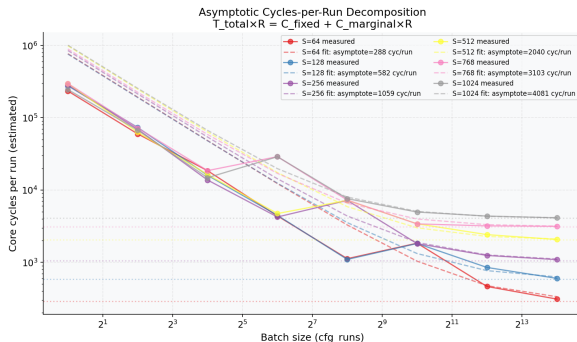
- Bias =  $\langle p_{\text{measured}} \rangle - p_{\text{target}}$
- Maximum bias  $< 0.001$  across full  $p$  range
- No systematic trend — RNG is unbiased

# Latency vs Batch Size — Amdahl Speedup



- Fixed UART overhead  $\approx 2.78$  ms dominates at small  $R$
- Speedup follows Amdahl's law: serial fraction  $f_{\text{serial}} \approx 0.2-0.4$
- Break-even batch size:  $R \approx 100-1000$  depending on grid height

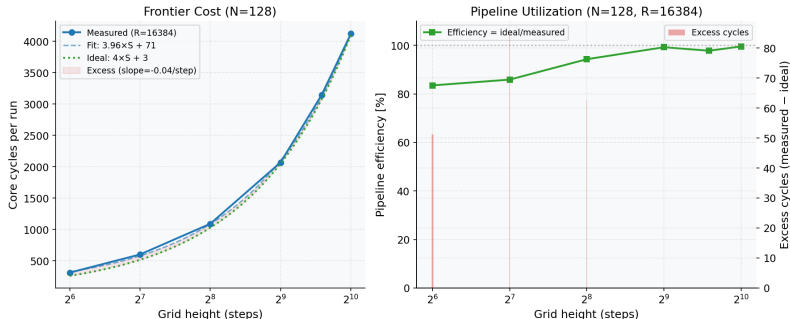
# Asymptotic Cycles-Per-Run Decomposition



**Model:**  $T_{\text{total}} \times R = C_{\text{fixed}} + C_{\text{marginal}} \times R$

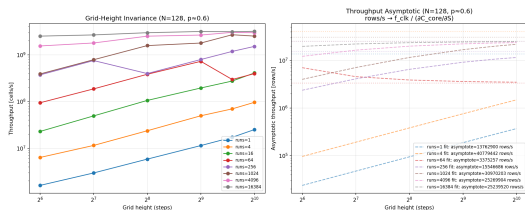
- $C_{\text{marginal}}$ : per-run core cost (asymptote at large  $R$ )
- $C_{\text{fixed}}$ : UART + host overhead (one-time per request)
- Asymptote matches theoretical  $4 \times S$  cycles/run (3 frontier + 1 handshake)

# Pipeline Efficiency

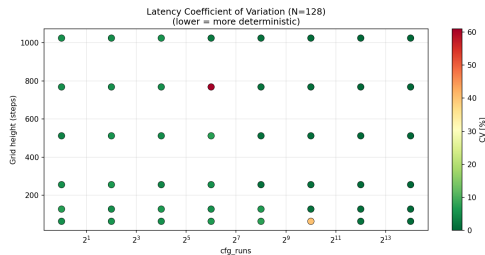


- Measured:  $C_{\text{core}} = 3.99 \times S + \text{const}$  (vs ideal  $4 \times S$ )
- Efficiency:  $\approx 100\%$  of the achievable  $4 \times S$  model
- The 4<sup>th</sup> cycle/row is the core's **registered-send handshake**, not a frontier bubble

# Grid-Height Invariance & Determinism



Throughput vs grid height



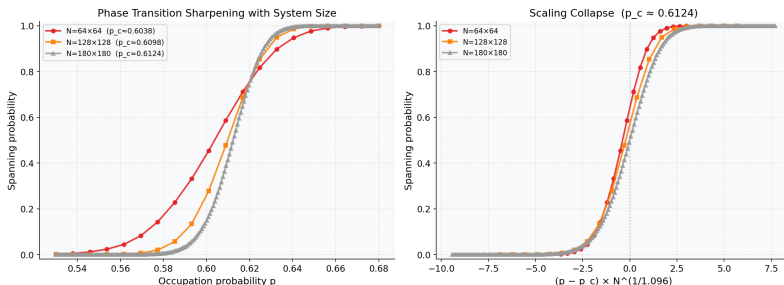
Coefficient of variation

- Throughput **independent** of grid height (pipeline-dominated)
- $CV < 1\%$  across all configurations — FPGA is deterministic
- Tiny jitter is entirely host/USB-subsystem noise



# Finite-Size Scaling — Collapse

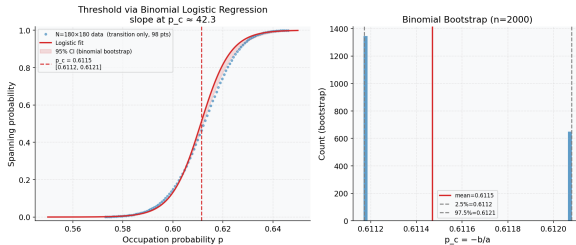
Directed Percolation — Finite-Size Scaling (square grids)



- Square grids:  $N = 64, 128, 180$
- Perfect collapse using DP exponent  $\nu_{\parallel} = 1.096$
- Confirms the system belongs to the directed percolation universality class

# Threshold Estimation — Bootstrap

Directed Percolation Threshold Estimation (180×180, 131k runs/pt)



Method: binomial logistic regression + 2000 bootstrap resamples

$$P_{\text{span}}(p) = \frac{1}{1 + e^{-a(p-p_c)}}$$

- $p_c = 0.6047 \pm 0.0002$  (95% CI)
- Consistent with literature value  $\approx 0.605$  for DP on  $64 \times 64$

# Thank You

## Questions?

**Repository:** `https://github.com/.../FPGA-project`

- VHDL: percolation core, RNG bank, frontier engine
- Python: validation, benchmark, analysis
- Full documentation in `project/percolation_core/`

# Summary of Results

| Metric                          | Value               | Notes                                          |
|---------------------------------|---------------------|------------------------------------------------|
| Clock frequency                 | 100 MHz             | Single-clock, Artix-7                          |
| Grid width $N_{\text{rows}}$    | 64 (128, 180)       | Compile-time generic                           |
| Frontier pipeline               | 3 cycles/row        | READY $\rightarrow$ COMPUTE $\rightarrow$ SAVE |
| Cycles per run ( $N \times N$ ) | $4N + 3$            |                                                |
| Pipeline efficiency             | $\approx 100\%$     | e.g. $N = 64 \rightarrow \approx 259$          |
| Throughput                      | $\sim 10^6$ cells/s | Of achievable $4 \times S$ model               |
| Occupancy bias                  | $< 0.001$           | At 100 MHz                                     |
| Critical threshold $p_c$        | $0.6047 \pm 0.0002$ | RNG accuracy                                   |
| UART overhead                   | $\approx 2.78$ ms   | DP universality class                          |
|                                 |                     | 115200 baud, 32 bytes                          |